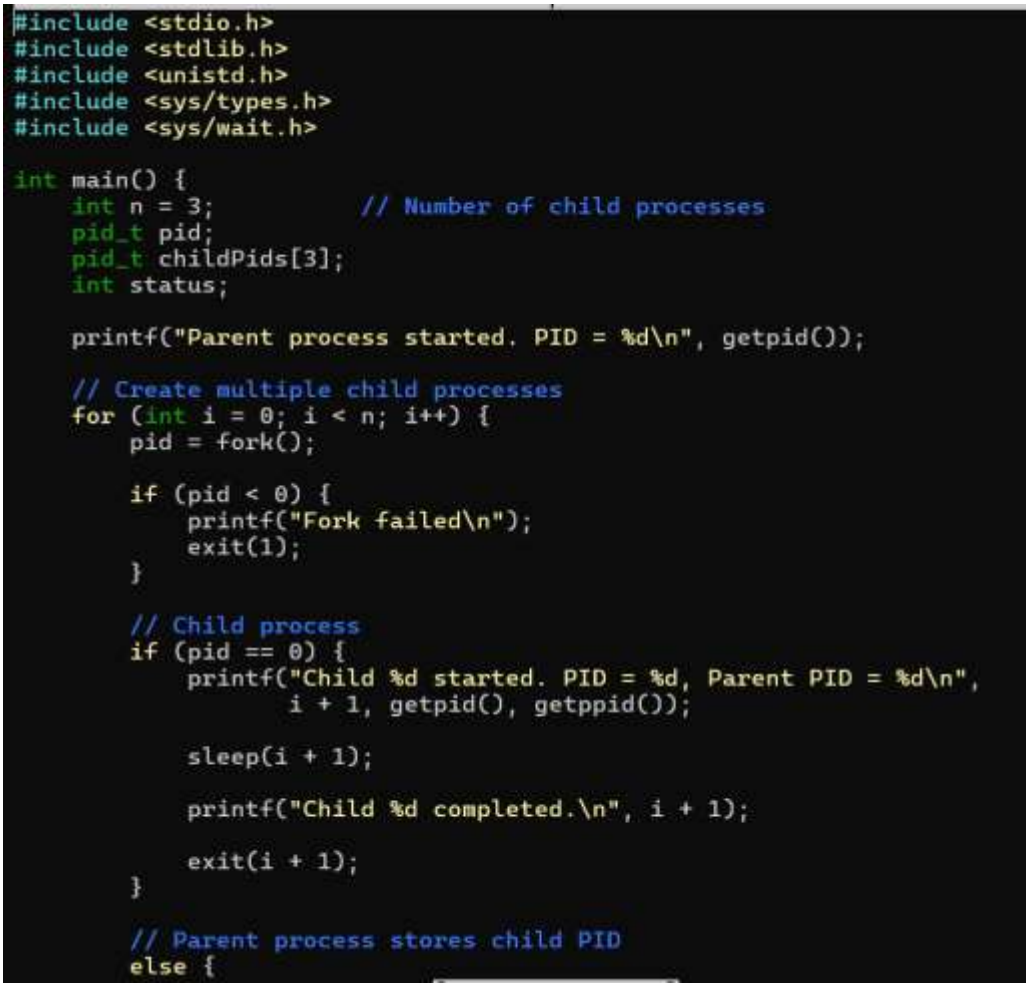
Practical – week4

1. **Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions**



```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int n = 3;          // Number of child processes
    pid_t pid;
    pid_t childPids[3];
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    // Create multiple child processes
    for (int i = 0; i < n; i++) {
        pid = fork();

        if (pid < 0) {
            printf("Fork failed\n");
            exit(1);
        }

        // Child process
        if (pid == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                   i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed.\n", i + 1);

            exit(i + 1);
        }

        // Parent process stores child PID
        else {
```

```c
    // Parent process stores child PID
    else {
        childPids[i] = pid;
    }
}

printf("\n--- Synchronization using wait() ---\n");

// wait() waits for ANY child process
for (int i = 0; i < n - 1; i++) {
    pid_t completedPid = wait(&status);

    if (completedPid > 0) {
        printf("Parent: Child with PID %d completed", completedPid);

        if (WIFEXITED(status)) {
            printf(" with exit status %d", WEXITSTATUS(status));
        }

        printf("\n");
    }
}

printf("\n--- Synchronization using waitpid() ---\n");

// waitpid() waits for a SPECIFIC child process
pid_t completedPid = waitpid(childPids[n - 1], &status, 0);

if (completedPid > 0) {
    printf("Parent: Specifically waited for child with PID %d",
           completedPid);

    if (WIFEXITED(status)) {
        printf(" with exit status %d", WEXITSTATUS(status));
    }
```

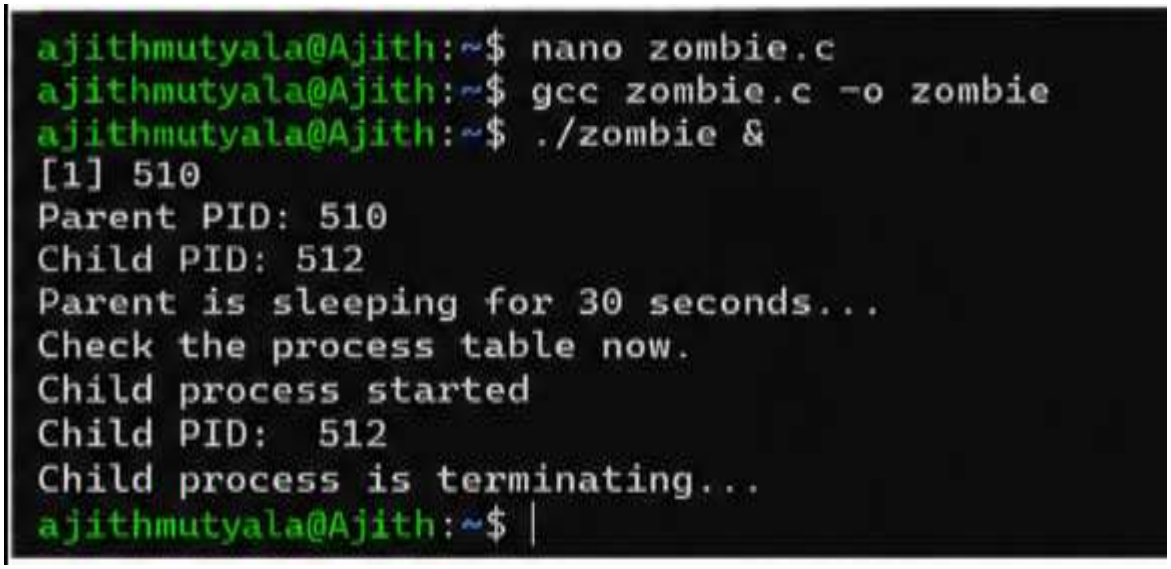
```
ajithmutyala@Ajith:~$ nano process.c
ajithmutyala@Ajith:~$ gcc process.c -o process
ajithmutyala@Ajith:~$ ./process
Parent process started. PID = 606
Child 1 started. PID = 607, Parent PID = 606
Child 2 started. PID = 608, Parent PID = 606

--- Synchronization using wait() ---
Child 3 started. PID = 609, Parent PID = 606
Child 1 completed.
Parent: Child with PID 607 completed with exit status 1
Child 2 completed.
Parent: Child with PID 608 completed with exit status 2

--- Synchronization using waitpid() ---
Child 3 completed.
Parent: Specifically waited for child with PID 609 with exit status 3

Parent process completed.
ajithmutyala@Ajith:~$
```

Report: I ran process.c which forked 3 child processes (PIDs 607, 608, 609) from parent PID 606. Using wait(), the parent synchronized with Child 1 and Child 2 as they finished (exit status 1 and 2), and with waitpid(), it specifically waited for Child 3 (exit status 3). All PIDs were distinct and valid, and the parent reported

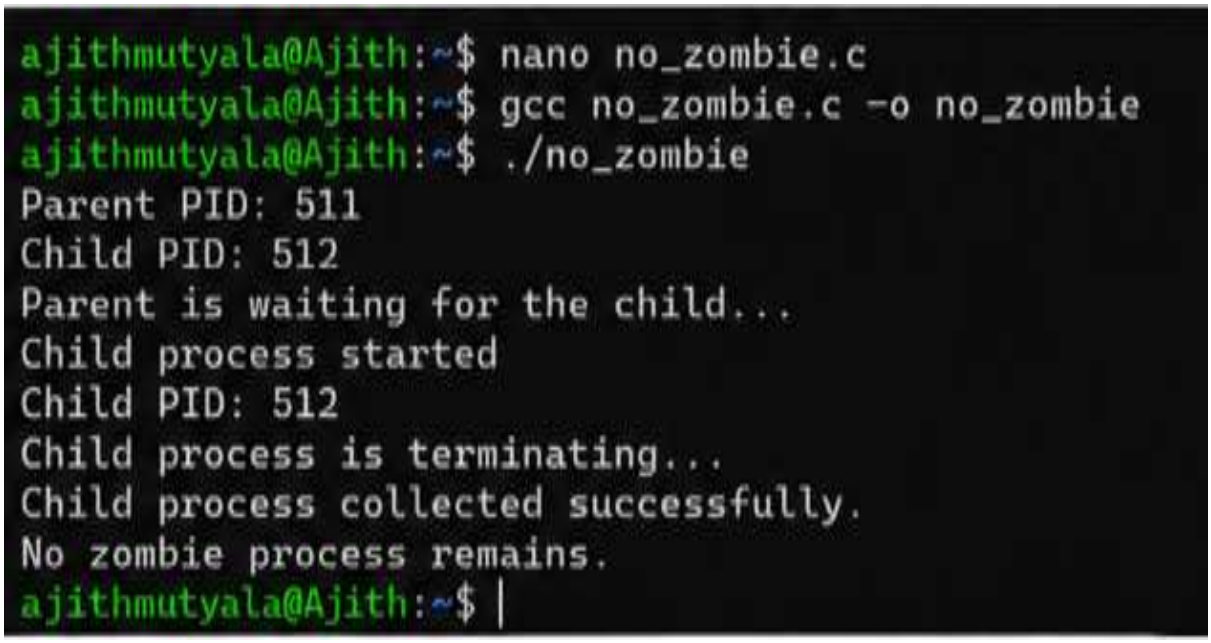
completion correctly after each child. This confirms that wait() and waitpid() both work as expected for process synchronization.

2. **Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.**

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child process started\n");
        printf("Child PID: %d\n", getpid());

        printf("Child process is terminating...\n");
        exit(0);
    }
    else {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is sleeping for 30 seconds...\n");
        printf("Check the process table now.\n");

        sleep(30);

        printf("Parent process completed.\n");
    }

    return 0;
}
```


ajithmutyala@Ajith:~$ nano zombie.c
ajithmutyala@Ajith:~$ gcc zombie.c -o zombie
ajithmutyala@Ajith:~$ ./zombie &
[1] 510
Parent PID: 510
Child PID: 512
Parent is sleeping for 30 seconds...
Check the process table now.
Child process started
Child PID: 512
Child process is terminating...
ajithmutyala@Ajith:~$

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child process started\n");
        printf("Child PID: %d\n", getpid());

        printf("Child process is terminating...\n");
        exit(0);
    }
    else {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for the child...\n");

        // Parent collects the child's exit status
        wait(NULL);

        printf("Child process collected successfully.\n");
        printf("No zombie process remains.\n");
    }

    return 0;
```



Report: I ran zombie.c, where the child terminates immediately but the parent sleeps for 30 seconds without calling wait(), causing the child (PID 512) to remain as a zombie in the process table until the parent finishes. Then I ran no_zombie.c, where the parent calls wait(NULL) right after forking, so it immediately collects the child's exit status (PID 512) and no zombie process remains. This confirms that a zombie process is created when a terminated child isn't reaped by the parent, and wait() prevents this by cleaning up the child right away.